

Aprendizaje de Máquina sobre estadísticas de venta de videojuegos en Steam

Santiago Alducin Villaseñor, 2026, A01707122

Abstract—El modelo de machine learning tiene como propósito determinar la calificación de un juego dependiendo diferentes factores como las reseñas, calificación de steam, cantidad de jugadores. Para esto primero conseguí una colección de datos de steam sobre diferentes juegos, procesé y limpié la información para el análisis, realicé un análisis exploratorio para encontrar relaciones entre variables y tendencias y generé el modelo de inteligencia artificial para predecir la calificación de un juego.

I. INTRODUCTION

Decidí utilizar una colección de juegos de steam ya que es un tema que me agrada y resulta de interés y consideré que podría ser divertido. Encontré una base de datos en github de steam del 2024 publicada por win7guru (2025), este es una serie de archivos .csv sobre información de los juegos, que pese a su gran cantidad de valores la información, esta carecía de calidad para un procesamiento sencillo.

II. ETL

El proceso de extracción de la información involucró varios pasos para limpiar la información y tenerla en un estado adecuado para el análisis y procesamiento posterior.

En primer lugar, la información no estaba normalizada, había columnas que contaban con valores de tipo JSON, html, o mal formateados con textos largos que se expandían múltiples columnas. Para hacer este primer procesamiento utilicé un script sencillo de Python que cambiara todas las comas del objeto json de precios por '|', para que el precio se mantuviera dentro de una misma celda. Posteriormente limpié manualmente de los diferentes archivos los textos malformados que se extendían, formateando la información a que se mantenga en su misma columna.

Ya que el archivo por lo menos tenía el mismo formato y columnas, pasé a leerlo utilizando pandas, donde tuve que especificar el formato ISO-8859-1 para que los caracteres especiales o de otros idiomas no causen que falle la lectura debido a títulos de juegos.

Ya con el dataset cargado empezó la limpieza de la información. Revisé la cantidad de instancias vacías en cada variable, y desgraciadamente el precio era una de las instancias con una mayor cantidad de variables vacías

(45.5%). Digo desgraciadamente porque consideraba que esta variable podía tener un gran impacto en las estadísticas de los juegos, y quería centrar mi análisis inicial en esta. Pero, ya que el dataset era suficientemente grande, consideré que puedo manejar la pérdida de información, y decidí eliminar las instancias que no contaban con precio, dejándome con una colección de 76 mil instancias prácticamente limpias. Igual que precios, durante los diferentes datasets encontré valores o columnas que decidí tirar porque no proveían de información relevante, como pueden ser las variables de tiempo de juego, que pese a que consideraba que iban a ser muy interesantes, resultaron inútiles debido a que estaban en 0. Como este caso hubo varios donde la variable se mantenía en todas las instancias o era un campo con valor 0 pero no vacío.

Como mencioné anteriormente, el campo de precios era aquél que venía en formato JSON, dificultando la lectura, entonces apliqué en la columna un regex para separar el precio del juego con su moneda, y luego convertí todos los precios a euros, ya que era la moneda más común en primer lugar.

app_id	name	release_date	is_free	price_currency	languages	type	price	currency
0	Counter-Strike	01/11/2000	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	4.19	EUR
1	Team Fortress Classic	01/04/1999	0	€/\$/¥/R\$	English+French+German+Italian+Spanish+...+Vietnamese	game	4.99	EUR
2	Day of Defeat	01/05/2000	0	€/\$/¥/R\$	English+French+German+Italian+Spanish+...+Vietnamese	game	4.99	EUR
3	Day of Defeat: Source	01/05/2000	0	€/\$/¥/R\$	English+French+German+Italian+Spanish+...+Vietnamese	game	4.99	EUR
4	Half-Life Opposing Force	01/11/1999	0	€/\$/¥/R\$	English+French+German+Italian+Spanish+...+Vietnamese	game	4.99	EUR
...	...	...	...	...	...	...	...	...
11000	Mazda Holloway	12/10/2024	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	1.19	EUR
11014	Sea Summer Camp	16/10/2024	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	4.12	EUR
110425	Potential Sea Adventures - Episode 5	12/10/2024	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	3.99	EUR
110602	NEURO	12/10/2024	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	0.75	EUR
110611	Holloway Stories: Written in Blood Collector's	12/10/2024	0	€/\$/¥/R\$	English+Chinese+...+Vietnamese	game	12.41	EUR

Luego leí la información de los demás archivos. De ellos varios eran categóricos con varios valores únicos que consideré no utilizar en la primera iteración del modelo, por lo que decidí conservar solo la información de reseñas de los juegos, calificación, cantidad de jugadores (rango probable), y cantidad de jugadores recientes. Aunque unos de estos sean categóricos utilizando el algoritmo de one-hot-encoding los transforme en columnas binarias para el análisis.

Y para terminar con esta primera fase de extracción y limpieza, estandaricé los campos numéricos continuos en relación a la cantidad de desviaciones estándar que tienen de la media.

Información limpia utilizada en el modelo

Variable	Tipo de dato	Transformación
Price eur	Float	standarización
Positive	Int	standarización
Negative	Int	standarización
Total	Int	standarización
Review score	int	Nada

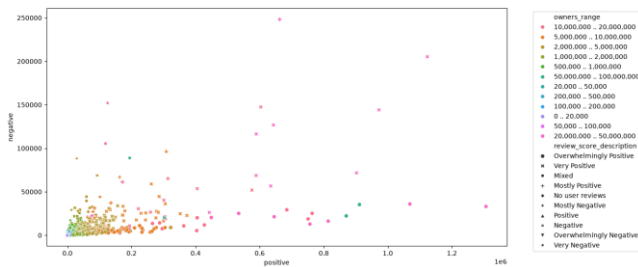
Concurrent users yesterday	Int	standarización
Owners_range	Class	One-hot-encoding
Review_score_description	class	One-hot-encoding

III. EDA

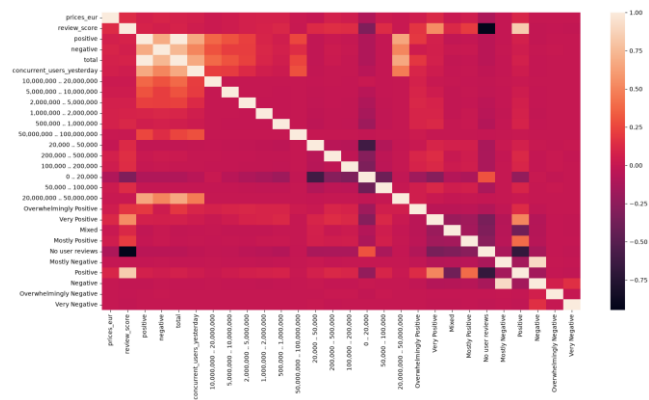
Ya que los datos están limpios y podemos trabajar con ellos de una mejor manera empieza un primer análisis para encontrar la relación que tienen las diferentes variables entre ellas.

En un primer vistazo y con conocimiento del funcionamiento del mercado digital de videojuegos, sabemos que entre más reseñas positivas la calificación del juego aumenta y que con las reseñas negativas esta disminuye. Entonces nuestro modelo debería de ser capaz de predecir la calificación de un juego con esta información.

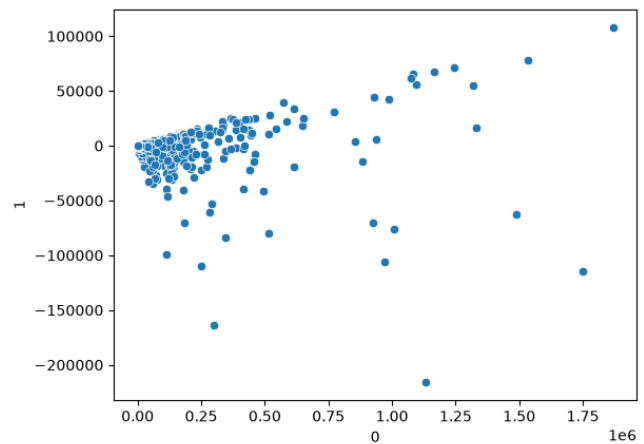
Para esto empecé solo con un gráfico que compare la cantidad de reseñas positivas y negativas con la cantidad de jugadores y la calificación que tiene. No sorprendentemente podemos ver que la cantidad de reseñas positivas tiene una relación directa con la calificación de los juegos y la cantidad de usuarios, y esta forma una especie de cono entre reseñas positivas y negativas. Hay una gran variación en los datos, pero se puede ver que entre más reseñas más probabilidad es que mucha gente lo tenga, y que sea de una buena calificación.



Luego creé una gráfica de calor para determinar las variables que tienen una mayor correlación, directa o inversa, de la cual se puede ver que el precio no tiene una relación fuerte con la calificación o usuarios de los juegos como planteaba en un primer momento. Pero, si hay una relación de la cantidad de reseñas positivas con negativas, cantidad de usuarios concurrentes, y calificaciones positivas.



Decidí tratar de utilizar un PCA para encontrar la tendencia, explorar un poco más la información y ver que puedo aprender de ella. Debido a que la gráfica de reseñas positivas y negativas me dio un cono, consideré que era buena idea probar para ver el resultado. En esta se puede ver una tendencia entre todas las variables lineal (85.8% de los datos), con una gran densidad de los datos cercana a 0. Esto facilitó aún más la decisión pues determiné que utilizar un modelo de regresión lineal para el modelo de Machine Learning sería una buena idea con una alta probabilidad de brindar resultados adecuados.



De esto, aunque no encontré la relación que buscaba en un inicio, considero que aún puede funcionar como un buen modelo para determinar la calificación de los juegos.

IV. FUNCIONAMIENTO DEL MODELO

python-ml.py

Como hemos visto en clase. El modelo es uno de regresión lineal utilizando el gradiente descendiente para minimizar el error. Este los separé en diferentes funciones que se encargan de partes específicas de la fórmula.

Menciono rápidamente antes de comenzar con la explicación del modelo: traté de “optimizar” el código empujando la mayor cantidad de operaciones posibles a numpy, ya que siendo escrito en C iba a hacer las operaciones más rápido.

Funciones:

- h : función hipótesis, saca el producto punto de todos los renglones con los parámetros de la iteración y les suma el bias.
- sum : Calcula el error promedio y calcula el error de los parámetros y bias.
- GD : Función principal del gradiente descendiente, manda a llamar la función anterior para calcular el error y calcula los nuevos valores para los parámetros.
- $train$: Se encarga de las iteraciones e hiperparámetros. En este inicializo los parámetros, los cuales intenté primero solo con random (Lo multipliqué por 100 para ver si cambiaba en algo cuanto tiempo tardaba en encontrar el mínimo, pero pues no hubo mucha diferencia)

V. RESULTADOS INICIALES

En la primera iteración completa que consideraré exitosa en el experimento, no logré un error de 0, pero llegue a uno de 0.21064, bastante pequeño, pero con unas varias iteraciones más creo que el modelo sería capaz de llegar a 0. Este duró 123 minutos en llegar a esta solución y su motivo de acabar fue que llegó al número máximo de épocas (20,000). Tomando en cuenta que tardó media hora en llegar a 1.5 de error y luego le tomó los otros 90 minutos en bajar 1.3 el error, llegar a 0 tomaría demasiado tiempo y considero que una mejor estrategia es pensar en maneras de optimizar el modelo para que pueda llegar a este mínimo más rápido.

La operación final que determinó el modelo es:

```
m = [ 1.00379146e-03 -3.85273582e+01 -
5.68627768e+00 4.24881234e+01
-7.19972018e-03 8.77256221e+00 2.82195938e+01
2.69690014e+01
2.64313150e+01 2.62696545e+01 6.78484363e+01
2.61474355e+01
2.61785088e+01 2.61513171e+01 2.61561085e+01
2.61486650e+01
3.33520907e+01 2.01116922e+00 1.00481652e+00
5.34317624e+01
-9.98210094e-01 4.84336975e+01 2.34600919e+00
5.54346460e+01
5.01124051e+01 9.66556352e+00 5.60390831e+00]
b = -74.60561428195325
```

Para facilitar el proceso de las pruebas y diagnóstico corrí nuevamente el modelo con 3,000 épocas, para tener resultados un poco más rápido ya conociendo el potencial que tendría el modelo.



Los resultados que se muestran arriba ya son los obtenidos con la última corrida del modelo. Estos serían los parámetros para determinar la calificación de un juego mediante los parámetros utilizados, con un error promedio final de:

Train loss MSE: 2.1787850651485887

Validation loss MSE: 4.533083859384745

Test loss MSE: 3.906969691261746

Resultado de validaciones:

Max error MSE: 6515.375404093403

Mean error MSE: 4.533083859384745

Mediana error MSE: 3.121822274148467e-05

Min error MSE: 7.657758837642246e-13

Resultados de pruebas:

Max error MSE: 1203.7602749952162

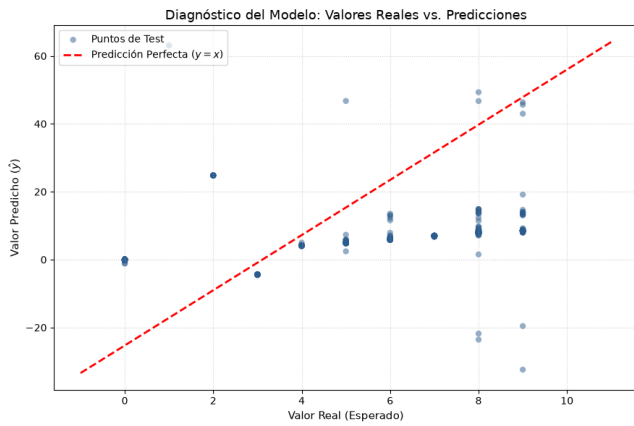
Mean error MSE: 0.5819433757903646

Mediana error MSE: 6.837892919988221e-06

Min error MSE: 1.0474713048729936e-12

VI. DIAGNÓSTICO

La gráfica de curva de aprendizaje entre entrenamiento y pruebas es consistente, mostrando que tenemos un modelo que se ajusta adecuadamente al problema lineal ya que no hay una desviación notable entre las dos y tienen un movimiento similar pese a trabajar con datos diferentes. Aunque de igual manera se puede notar que validación y pruebas tienen cierto grado de overfitting, con una diferencia promedio entre ellas de .6% y una diferencia de ~2% con training.



Haciendo una gráfica de la predicción con los resultados esperados, se puede ver que la mayoría de predicciones si se encuentra en el rango esperado cerca del valor esperado, pero hubo instancias que resultaron en valores atípicos fuera del rango de valores esperados, mostrando que es bastante sensible y una mala predicción puede resultar en resultados fuera de lo normal.

Gracias a los resultados podemos pensar que se tiene un bajo sesgo en el modelo, ya que una gran parte de los datos tiene un error cercano a 0, teniendo un MSE de 0.5 y una media de 6e-6.

Desviación estándar: 0.5288050325623437

T-Student: -0.04485415635173457

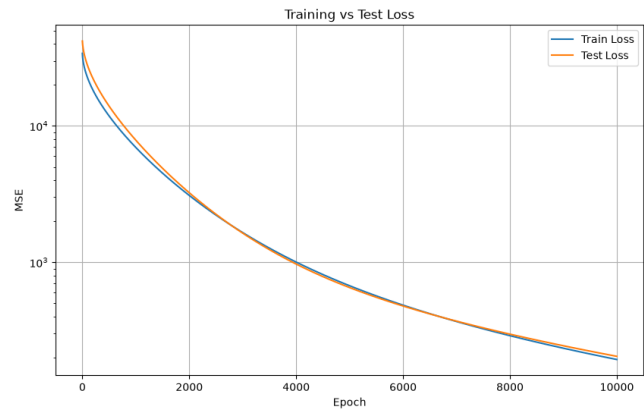
VII. Mejora

steam-project-huber-nestrov.py

El modelo está teniendo buenos resultados, la mejora es constante y el error bajó bastante, pero los mejores resultados que he tenido es dejando el modelo correr durante 2 horas y permitir que continúe disminuyendo el error indefinidamente. Pero, este acercamiento toma bastante tiempo y es indefinido, por lo cual considero que si pongo un learning rate adaptivo podré cambiar el tamaño de paso cuando este sea insuficiente y el modelo disminuya en velocidad de disminución del error.

Mi teoría inicial es que si aumento el learning rate conforme el modelo avanza entonces el tamaño de paso será mayor y el error disminuirá más rápido, pero para evitar que el paso sea demasiado grande, igual es necesario añadir ciertos límites y estrategias para disminuir el tamaño de paso. De esta manera, el modelo debería tener una caída del error rápida durante más tiempo y se podrá tener mejores resultados en la misma cantidad de tiempo.

Una estrategia que identifiqué es Adagrad (CodeSignal, s.f.), que adapta el alfa dividiendo su valor por la raíz del acumulado del gradiente (+ un valor para evitar dividir entre 0). Este experimento resulto contraproducente, ya que el error disminuyó menos que sin el alfa adaptativa



Propuesta de mejora dos: Cambiar la fórmula de error de MSE a Huber Loss, que mezcla MAE con MSE para tener una mejor sensibilidad al calcular el error. Si el error es mayor a un delta se utiliza MSE, si es menor se utiliza $error = \delta \times (|error| - \frac{\delta}{2})$ (GeeksforGeeks, 2025). E igual implementé Nesterov Momentum (Brownlee, 2021), este para que el modelo corra más rápido y no tarde tanto en llegar a un error bajo. Este funciona agregando una variable extra que determina el momentum, esto se logra calculando el cambio de la variable y utilizando su resultado para determinar su nuevo valor.



Resultados de validación:

Max error MSE: 8602.953550147662

Mean error MSE: 6.129573975710204

Mediana error MSE: 1.1913600050684335e-06

Min error MSE: 1.057033830298491e-13

Max error Huber: 138.00316209463932

Mean error Huber: 0.13082613494493678

Mediana error Huber: 5.956800025342167e-07

Min error Huber: 5.285169151492455e-14

Como se puede ver en la gráfica, la disminución del error ya no es tan suave como en la ecuación de gradiente descendiente original, teniendo una caída menos uniforme. El error MSE me dio mayor al modelo original, con errores en prueba de:

Max error MSE: 8606.546034862851

Mean error MSE: 9.233833406691048

Mediana error MSE: 1.2343527398985217e-06

Min error MSE: 2.9137590713303366e-13

De igual manera muestro el error de Huber promedio:

Max error Huber: 138.03220814403187

Mean error Huber: 0.18067411849038323

Mediana error Huber: 6.171763699492609e-07

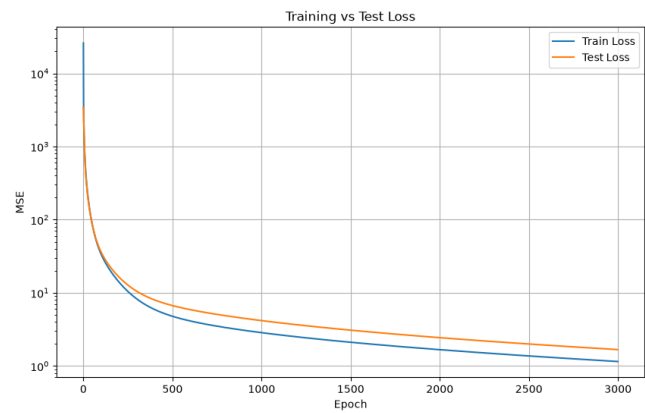
Min error Huber: 1.4568795356651683e-13

Tomando en comparación la versión del modelo original, se ve que el modelo tiene un mayor error que usando el estándar de error, pero de igual manera, cabe mencionar que este resultado lo obtuvo en una menor cantidad de iteraciones y tiempo que el modelo original habiendo terminado a los 24 minutos.

Para una mejor comparativa correré ambos modelos de manera cruzada, el original con un máximo de épocas de 3,000; y el Huber-Nesterov por 2 horas para comparar los resultados. Si el nuevo modelo no muestra una mejora considerable en comparación al antiguo se podrá concluir que por lo menos estas dos técnicas de optimización no ayudaron en el modelo de regresión lineal con el que estamos trabajando.

El modelo original corrió las 3000 instancias en 17 minutos, pero, pero su error promedio es mayor que en la mejora por ~1.1% en pruebas y

Prueba MSE promedio: 1.6789447112152602



Corriendo el nuevo modelo durante 20,000 iteraciones, el programa tardó casi 3 horas, tomando más tiempo que el modelo original, pero su error resultante fue:



Max error MSE: 1168.531812710378

Mean error MSE: 0.5532986103447977

Mediana error MSE: 7.245170674102352e-08

Min error MSE: 3.045834032510846e-14

Max error Huber: 50.150691888051114

Mean error Huber: 0.034748310058320846

Mediana error Huber: 3.622585337051176e-08

Min error Huber: 1.522917016255423e-14

Comparándolo con la reiteración del modelo a 20 mil épocas, la mejora dio un ínfimo mejor resultado por 50% más de tiempo, por lo que concluiría que esta versión sirve para hacer iteraciones más rápidas y sirve para hacer un mejor modelo.

VIII. MODELO CON FRAMEWORK

steam-project-fw.py

Para el modelo con framework cambiaré de interés, prediciendo la categoría a la que pertenece el juego / calificación de los usuarios de acuerdo con sus reseñas.

Este me dio una exactitud del 84.72% y poniéndolo a prueba de 10 instancias tuvo 8 correctas, una se equivocó bastante y otra fue un error leve.

	Clase Esperada	Clase Predicha	¿Correcto?
0	Mostly Negative	Overwhelmingly Positive	False
1	Mixed	Mixed	True
2	Positive	Positive	True
3	Positive	Positive	True
4	Very Positive	Very Positive	True
5	No user reviews	No user reviews	True
6	Very Positive	Very Positive	True
7	No user reviews	No user reviews	True
8	No user reviews	No user reviews	True
9	Mostly Positive	Positive	False

Referencias

CodeSignal. (s. f.). *Adaptive learning rate methods*. CodeSignal Learn. Recuperado el 3 de septiembre de 2026, de <https://codesignal.com/learn/courses/foundations-of-optimization-algorithms/lessons/adaptive-learning-rate-methods>

GeeksforGeeks. (23 de julio de 2025). One-Hot-Encoding from a Pandas Column Containing a List. <https://www.geeksforgeeks.org/machine-learning/one-hot-encoding-from-a-pandas-column-containing-a-list/>

Win7guru. (2025, enero 1). Steam-dataset-2024. Github. <https://github.com/win7guru/steam-dataset-2024>

GeeksforGeeks. (1 de agosto de 2025). *Huber Loss Function in Machine Learning*. <https://www.geeksforgeeks.org/machine-learning/huber-loss-function-in-machine-learning/>